Object Oriented Programming (CSEG1044)
Unit 03 – Lecture 01 Notes
Nested Classes (Types + Practical Use-Cases)

Tofik Ali

February 27, 2026

Contents

1 How to use these notes (self-study) **2**

2 Syllabus Alignment (Unit 03) **2**

3 Prerequisites **2**

4 Learning Outcomes **2**

5 Lecture Overview **2**

6 Key Terms (Quick) **2**

7 Detailed Notes **3**

7.1 Nested classes at a glance . . . 3
7.2 Motivation: scope and encapsulation . . . 3
7.3 Types of nested classes (with intuition) . . . 3
7.3.1 1) Static nested class . . . 3
7.3.2 2) Inner class (non-static member class) . . . 4
7.3.3 3) Local class (inside a method) . . . 4
7.3.4 4) Anonymous class . . . 4
7.4 Access rules (important and often surprising) . . . 5
7.5 Common mistakes (and quick fixes) . . . 5

8 Worked Example (tutorial): one outer type using all 4 kinds **5**

9 In-Class Exercises (with brief solutions) **7**

10 Self-Study Practice (with hints) **7**

11 Summary (1 minute) **7**

12 Checkpoint Questions (with sample answers) **7**

1 How to use these notes (self-study)

Read in order. Try the tiny code snippets mentally first, then run the worked example. For each section, answer the checkpoint questions before you move on.

2 Syllabus Alignment (Unit 03)

Unit 03: nested classes and their types

3 Prerequisites

You should already be comfortable with:

- classes/objects and access modifiers (`private/public`),
- interfaces and implementations,
- basic method calls and constructors.

4 Learning Outcomes

After this lecture, you should be able to:

- Explain why nested classes exist and when they improve design.
- Differentiate static nested, inner, local, and anonymous classes.
- Recognize common use cases like helpers and callbacks/listeners.

5 Lecture Overview

A **nested class** is a class declared **inside** another class, or inside a method. Java supports nested classes because sometimes a type is only meaningful within a bigger type. Nested classes let you:

- keep helpers **close** to where they are used (readability),
- hide implementation details (encapsulation),
- implement small callbacks/listeners without creating many separate files.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

6 Key Terms (Quick)

- **Static nested class**: nested class with `static`; no implicit outer instance.
- **Inner class**: non-static member class; has implicit reference to outer object.

- **Local class**: class declared inside a method.
- **Anonymous class**: unnamed local class used inline to implement an interface/extend a class.
- **Callback**: passing code/behavior to run later (often via interfaces).
- **Effectively final**: a local variable that is not reassigned after initialization.
- **Outer instance**: the object of the outer class that an inner class "belongs to".

7 Detailed Notes

7.1 Nested classes at a glance

Use this table when you are unsure which kind you need:

Type	Needs outer object?	Typical use-case
Static nested	No	helper tied to a concept (e.g., `Builder`, `Entry`, `Node`)
Inner (member)	Yes	helper that needs outer state (iterator, view, controller)
Local (in method)	No (but can capture)	short helper for one method, but named
Anonymous	No (but can capture)	one-time callback/listener implementation

7.2 Motivation: scope and encapsulation

Sometimes a helper type is only meaningful inside one class. If you declare it as a top-level class, it becomes visible everywhere and increases the public "API surface". By nesting it, you communicate: *this helper belongs here.*

7.3 Types of nested classes (with intuition)

7.3.1 1) Static nested class

- Declared with `static`.
- Does **not** need an outer object to be created.
- Can only directly access **static** members of the outer class.

Use-case: helper/utility class tied to a concept (e.g., `Map.Entry`, `Builder`, `Node`).

Mini example (validator helper).

```
class BankAccount {
    private double balance;

    static class Validator {
        static void requirePositive(double amount) {
            if (amount <= 0) throw new IllegalArgumentException("amount");
```

```
        }
    }

    void deposit(double amount) {
        Validator.requirePositive(amount);
        balance += amount;
    }
}
```

Notice: the validator does not need an outer object, so it is static.

7.3.2 2) Inner class (non-static member class)

- Has an implicit reference to the outer object.
- Can access outer instance fields/methods directly.

Use-case: iterators or helpers that need access to the outer object's state.

How to create an inner class object (syntax). An inner class instance is tied to an outer instance:

```
Outer outer = new Outer();
Outer.Inner inner = outer.new Inner();
```

Important design note. Because an inner class carries an outer reference, be careful when the inner object may live longer than the outer object (can keep it in memory longer than expected). If you do not need outer state, prefer a static nested class.

7.3.3 3) Local class (inside a method)

- Exists only within the method scope.
- Can access local variables only if they are **effectively final**.

Use-case: small helper that is too specific to become a member class.

Why effectively final? When a local/anonymous class uses a local variable, the compiler "captures" its value. If that value could change later, the captured value would be confusing/unsafe. So Java requires the variable not to be reassigned.

7.3.4 4) Anonymous class

- Created inline with `new Interface() { ... }`.
- Useful for one-time implementations (callbacks/listeners).

In modern Java, lambdas often replace anonymous classes for functional interfaces (Unit 04), but anonymous classes are still important to understand because:

- older code uses them frequently,

- you can create an anonymous class even for non-functional interfaces and abstract classes.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

7.4 Access rules (important and often surprising)

- The outer class can access private members of the nested class.
- The nested class can access private members of the outer class.

This is allowed because the Java compiler generates the necessary bridge code. So `private` is still respected at the source-code level, but nested/outer types are considered closely related.

7.5 Common mistakes (and quick fixes)

1. **Mistake:** Trying to create an inner class without an outer object.

   Fix: create an outer instance first: `outer.new Inner()`.

2. **Mistake:** Local/anonymous class uses a variable that you later modify.

   Fix: keep the variable effectively final (do not reassign), or move state into a field.

3. **Mistake:** Choosing an inner class when a static nested class is enough.

   Fix: if you do not need outer instance data, make it `static`.

8 Worked Example (tutorial): one outer type using all 4 kinds

We will build a small `IntBag` container and use nested classes for iteration, helpers, and filtering.

Step 1: Define a small callback interface

```
interface IntPredicate {
    boolean test(int x);
}
```

This lets us pass "a condition" (behavior) into a method like `countIf(...)`.

Step 2: Outer class + inner cursor (needs outer state)

```
import java.util.Arrays;

public class IntBag {
    private final int[] data;

    public IntBag(int[] data) {
        this.data = data.clone();
    }

    // INNER class: it needs access to 'data'
    public class Cursor {
```

```java
        private int idx = 0;
        public boolean hasNext() { return idx < data.length; }
        public int next() { return data[idx++]; }
    }

    public Cursor cursor() { return new Cursor(); }

    public int countIf(IntPredicate p) {
        int c = 0;
        for (int x : data) if (p.test(x)) c++;
        return c;
    }

    @Override public String toString() {
        return Arrays.toString(data);
    }
}
```

Why inner? `Cursor` directly reads `data` from the outer object.

Step 3: Static nested helper (does not need outer object)

```java
public class IntBag {
    // ... keep previous code ...

    public static class Stats {
        public static int max(int[] a) {
            int m = Integer.MIN_VALUE;
            for (int x : a) if (x > m) m = x;
            return m;
        }
    }
}
```

Why static? `Stats.max` only needs an array; it does not need an `IntBag` instance.

Step 4: Local class and anonymous class (filters)

```java
public class Demo {
    public static void main(String[] args) {
        IntBag bag = new IntBag(new int[]{3, 1, 2, 8, 5});
        System.out.println("Bag = " + bag);

        // INNER class object comes from an outer object
        IntBag.Cursor cur = bag.cursor();
        System.out.print("Iterate: ");
        while (cur.hasNext()) System.out.print(cur.next() + " ");
        System.out.println();

        // LOCAL class (named helper inside a method)
        final int threshold = 3; // effectively final
        class GreaterThan implements IntPredicate {
            public boolean test(int x) { return x > threshold; }
```

```
        }
        System.out.println("count(x > 3) = " + bag.countIf(new GreaterThan()));

        // ANONYMOUS class (one-time implementation)
        int evens = bag.countIf(new IntPredicate() {
            public boolean test(int x) { return x % 2 == 0; }
        });
        System.out.println("count(even) = " + evens);
    }
}
```

Later in Unit 04 you will replace many anonymous classes like the even-check with a lambda, but the idea (passing behavior) remains the same.

9 In-Class Exercises (with brief solutions)

1. Which nested type has an implicit reference to the outer object? **Answer:** inner class.
2. Which nested type is often used for callbacks/listeners? **Answer:** anonymous class.
3. Can a local class access a non-final local variable? **Answer:** no (must be effectively final).

10 Self-Study Practice (with hints)

1. Add a method `sum()` to `IntBag`. Keep it simple.

   Hint: loop over `data`.

2. Modify the demo to count odd numbers using an anonymous class.

   Hint: `x % 2 != 0`.

3. Make a new static nested class `MinMax` that computes both min and max.

   Hint: return an `int[]` of length 2 or create a tiny nested data class.

11 Summary (1 minute)

- Use a **static nested class** when the helper does not need an outer object.
- Use an **inner class** when the helper needs outer state.
- Use a **local class** when the helper is only for one method but benefits from a name.
- Use an **anonymous class** for one-time callbacks (often replaced by lambdas later).

12 Checkpoint Questions (with sample answers)

Q1. Why do nested classes help encapsulation?

Sample answer: They keep helper types close to where they are used and reduce the public API. This prevents accidental use from unrelated code and improves readability.

Q2. What is the difference between a static nested class and an inner class?

Sample answer: A static nested class does not have an implicit outer reference and can be created without an outer object. An inner class has an implicit reference to an outer instance and can access its instance members.